Python Cursus: Week 2

Leerdoelen

- ☐ Herkennen van verschillende data-structuren.
- ☐ Leren werken en toepassen van vector-operaties in Python.
- ☐ Functies, doc-strings, opties & plotten.

Veiligheid

We gaan programmeren, hier zijn geen veiligheidskwesties aan gebonden. Probeer alleen zoveel mogelijk zelfstandig te programmeren. Gebruik ChatGPT verstandig:

1. Gebruik het als uitlegmachine, niet als codegenerator
2. Gebruik het om eigen code te laten verbeteren, maar niet herschrijven
3. Gebruik het niet voor volledige oplossingen, snap je iets niet? Vraag het de docent.

Opdracht: Week 2 in één Jupyter Notebook

Maak een nieuw Jupyter Notebook aan met de naam `week2.ipynb`. Voer alle onderstaande onderdelen uit als losse cellen in dit notebook.

Deel 1: Datastructuren

Q1

Maak in `week2.ipynb` een eerste codecel waarin je voorbeelden maakt van (Link):

a) een lijst;
b) een array;
c) een dictionary.

Oplossing:

Cel 1 – voorbeelden datastructuren

```
# Voorbeelden van datastructuren
lijst = [1, 2, 3, 4]
dictionary = {"a": 1, "b": 2, "c": 3}
array = np.array([1,2,3,4])

print("Lijst:", lijst)
print("Dictionary:", dictionary)
print("Array:", array)
```

Q2

a) Maak een tweede codecel waarin je in commentaar (`# ...`) kort noteert wanneer je welke datastructuur zou gebruiken (bijv. lijst vs. dictionary).
b) Bereken het kwadraat van de waardes in de `lijst`.
c) Bereken het kwadraat van de waardes in de NumPy-`array`.
d) Wat gebeurt er als je een lijst vermenigvuldigt met een integer?
e) Waarvoor zou je een `dictionary` kunnen gebruiken?

Oplossing:

a) Je gebruikt een lijst om dingen in op te slaan (net als een boodschappenlijstje). Dit kunnen dus **strings**, **integers**, etc. zijn. Een array (NumPy-array) is bedoeld voor numerieke berekeningen; de elementen zijn typisch **floats** en **integers** en je kunt er efficiënt wiskundige operaties mee uitvoeren. Een dictionary is een soort bibliotheek: je haalt elementen op aan de hand van een *key*. Dit wordt minder vaak gebruikt binnen de basis-natuurkunde, maar is handig voor gestructureerde data.

b) **Cel – kwadraat van de lijst**

```
lijst = [1, 2, 3, 4]
lijst_kwadraat = [i**2 for i in lijst]

print("Lijst:", lijst)
print("Lijst in het kwadraat:", lijst_kwadraat)
```

c) **Cel – kwadraat van de array**

```
import numpy as np

array = np.array([1, 2, 3, 4])
array_kwadraat = array**2  # vectoriële operatie

print("Array:", array)
print("Array in het kwadraat:", array_kwadraat)
```

d) Als je een integer probeert te vermenigvuldigen met een lijst, dan wordt de lijst herhaald in plaats van elementgewijs vermenigvuldigd.
   Bijvoorbeeld: `3 * [1, 2, 3]` resulteert in `[1, 2, 3, 1, 2, 3, 1, 2, 3]`.
   Python voert dus geen wiskundige vermenigvuldiging uit per element — het ziet de operatie als “lijst drie keer achter elkaar plakken”.
e) Een dictionary kun je bijvoorbeeld gebruiken om:
   - meetdata per sample-ID op te slaan, bijv. `metingen = {"sample1": [1.2, 1.3], "sample2": [0.9, 1.0]}`
   - fysische constanten te bewaren, bijv. `constanten = {"g": 9.81, "c": 3.00e8}`
   - experimentele instellingen vast te leggen, bijv. `config = {"temperatuur": 293, "druk": 1.0}`.

Deel 2: Vector-operaties met `numpy`

Q3

Maak een nieuwe codecel waarin je `numpy` importeert en twee vectoren definieert.

Oplossing:

Cel 3 – basis vector-operaties

```
import numpy as np

v1 = np.array([1, 2, 3, 4, 5])
v2 = np.array([5, 4, 3, 2, 1])

print("v1 =", v1)
print("v2 =", v2)
```

Q4

Bereken het product (elementsgewijs) van de vectoren.

Oplossing:

Cel – elementgewijs product van twee vectoren

```
import numpy as np

v1 = np.array([1, 2, 3, 4, 5])
v2 = np.array([5, 4, 3, 2, 1])

# Elementgewijs product
element_product = v1 * v2

print("v1 * v2 =", element_product)
```

Q5

Bereken het dotproduct tussen de vectoren.

Oplossing:

Cel – dotproduct van twee vectoren

```
import numpy as np

v1 = np.array([1, 2, 3, 4, 5])
v2 = np.array([5, 4, 3, 2, 1])

# Dotproduct (inwendig product)
dot = np.dot(v1, v2)

print("dot(v1, v2) =", dot)
```

Q6

Bereken het dotproduct tussen dezelfde twee vectoren, maar dit keer zonder `numpy` — gebruik gewone Python-lijsten en een `for`-loop.

Oplossing:

Cel – dotproduct met lijsten en for-loops

```
# Dotproduct berekenen met lijsten en een for-loop

v1 = [1, 2, 3, 4, 5]
v2 = [5, 4, 3, 2, 1]

dot = 0
for i in range(len(v1)):
    dot += v1[i] * v2[i]

print("Dotproduct (met for-loop) =", dot)
```

Q7

Maak een extra codecel waarin je de norm (lengte) van een vector berekent, en bereken ook de hoek tussen twee vectoren. Hint! De norm van de vector wordt veel gebruikt en heeft binnen numpy een standaard functie. Link. Verder wordt de hoek tussen twee vectoren weergeven als $\cos(\theta) = \frac{\vec{v_1} \cdot \vec{v_2}}{|v_1||v_2|} =$

Oplossing:

Cel 3 – norm en hoek tussen twee vectoren

```
# Norm (lengte) van beide vectoren
norm_v1 = np.linalg.norm(v1)
norm_v2 = np.linalg.norm(v2)

print("||v1|| =", norm_v1)
print("||v2|| =", norm_v2)

# Hoek tussen de twee vectoren (in radialen en graden)
cos_theta = np.dot(v1, v2) / (norm_v1 * norm_v2)
theta_rad = np.arccos(cos_theta)
theta_deg = np.degrees(theta_rad)

print("cos(theta) =", cos_theta)
print("theta (rad) =", theta_rad)
print("theta (deg) =", theta_deg)
```

Deel 3: Functies, docstrings & plotten

Docstrings. Een docstring is een korte beschrijving die je *in* een functie zet om uit te leggen wat deze doet, welke parameters de functie verwacht en wat de functie teruggeeft. In Python plaats je een docstring direct onder de functiedefinitie tussen drie dubbele aanhalingstekens: `""" ... """`. Tools zoals VSCode kunnen deze docstrings automatisch tonen wanneer je met de muis over een functie hovert.

VSCode-extension. De extensie die ondersteuning biedt voor docstrings en functie-informatie is **Pylance**. Deze zorgt voor slimme hints, type-informatie en toont automatisch de docstring van jouw functies én van Python-bibliotheken.

Auto-docstring. Voor het automatisch genereren van docstrings bestaat de extensie **autoDocstring**. Wanneer je in een functie `"""` typt en op `Enter` drukt, vult deze extensie automatisch een docstring-sjabloon in met secties voor beschrijving, parameters en returnwaarden. Dit helpt je om consistente, duidelijke documentatie voor al je functies te schrijven.

Q8

Maak een nieuwe codecel waarin je een functie definieert met een docstring die een vector elementgewijs tot een bepaalde macht verheft.

Oplossing:

Cel – functie met docstring

```python
import numpy as np

def macht(vector, power=2):
    """
    Verhoogt elk element van een vector tot de gegeven macht.

    Parameters
    ----------
    vector : array-like
        Invoervector (lijst of numpy-array).
    power : int of float, optioneel
        De macht waarmee elk element wordt verheven (standaard: 2).

    Returns
    -------
    numpy.ndarray
        Vector waarbij elk element is verheven tot 'power'.
    """
    vector = np.array(vector)
    return vector**power

print(macht([1, 2, 3]))           # gebruikt standaard power=2
print(macht([1, 2, 3], power=3))  # gebruiker kiest zelf de macht
```

Q9

Ik wil graag dat deze functie een standaardwaarde van 2 meegeeft aan de macht; als gebruiker heb ik dan een optie om deze aan te passen. Leg uit hoe dit werkt.

Oplossing: Dit gebeurt via de parameterdefinitie `power=2`. Als de gebruiker niets invult, wordt `power` automatisch 2. Wanneer de gebruiker zelf een waarde meegeeft, bijvoorbeeld `power=5`, dan overschrijft dit de standaardwaarde en wordt de vector tot die macht verheven.

Q10

Waar ben je in het verleden nog meer `opties` (default parameters) tegengekomen?

Oplossing: Opties (standaardwaarden) zag je eerder bij functies zoals:

- `print(..., sep=" ", end="")` — `sep` en `end` hebben standaardwaarden.

- `np.linspace(start, stop, num=50)` — `num` heeft een standaardwaarde.
- `plt.plot(x, y, label=None)` — veel plotopties zijn optioneel.

Q11

Maak een nieuwe codecel waarin je een eenvoudige plot (Link) maakt van de functie $f(x) = x^2$ op het interval $[-3, 3]$.

Oplossing:

Cel 6 – plot van f(x) = x2**

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-3, 3, 200)
y = x**2

plt.plot(x, y)
plt.xlabel("x")
plt.ylabel("f(x) = x^2")
plt.grid(True)
plt.show()
```

Q12

Maak een codecel waarin je een plot maakt met twee y-assen: op de linker-as $f(x) = x^2$ en op de rechter-as $g(x) = \sin(x)$, beide uitgezet op hetzelfde x-interval.

Oplossing:

Cel – plot met twee y-assen

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-3, 3, 400)
y1 = x**2          # linker y-as
y2 = np.sin(x)     # rechter y-as

fig, ax1 = plt.subplots()

# Linker y-as
ax1.plot(x, y1, color="blue")
ax1.set_xlabel("x")
ax1.set_ylabel("f(x) = x^2", color="blue")
ax1.tick_params(axis="y", labelcolor="blue")

# Rechter y-as
ax2 = ax1.twinx()
ax2.plot(x, y2, color="red")
ax2.set_ylabel("g(x) = sin(x)", color="red")
ax2.tick_params(axis="y", labelcolor="red")

plt.title("Plot met twee y-assen")
plt.grid(True)
plt.show()
```

Q13

Maak een codecel waarin je een **log–log plot** maakt van $f(x) = x^3$ op het interval $[0.1, 10]$.

Oplossing:

Cel – log–log plot van f(x) = x3**

```python
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0.1, 10, 400)   # kleinste waarde > 0 voor log-schaal
y = x**3

plt.loglog(x, y)
plt.xlabel("x (log-schaal)")
plt.ylabel("f(x) = x^3 (log-schaal)")
plt.grid(True, which="both")
plt.show()
```

Q14

Maak een codecel waarin je twee functies in **subplots** (Link) plaatst: boven $f(x) = \sin(x)$ en onder $g(x) = \cos(x)$, beide op het interval $[0, 2\pi]$.

Oplossing:

Cel – subplots met sin en cos

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 2*np.pi, 400)

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(6, 6), sharex=True)

# Bovenste subplot
ax1.plot(x, np.sin(x), color="blue")
ax1.set_ylabel("sin(x)")
ax1.grid(True)

# Onderste subplot
ax2.plot(x, np.cos(x), color="red")
ax2.set_ylabel("cos(x)")
ax2.set_xlabel("x")
ax2.grid(True)

plt.tight_layout()
plt.show()
```

Appendix 1: data-types

Python werkt met verschillende soorten gegevens, zogenaamde *datatypes*. De belangrijkste basistypen zijn:

`int`
: Een geheel getal (integer), bijvoorbeeld `3`, `-17`, `42`. Wordt gebruikt voor tellingen, indexen en numerieke berekeningen zonder decimalen.

`float`
: Een decimaal getal (floating point), bijvoorbeeld `3.14`, `0.001`. Dit datatype gebruik je voor fysische grootheden, metingen en elke numerieke berekening waarin decimalen voorkomen.

`str`
: Een tekststring (string), bijvoorbeeld `"Hallo"`, `"3.14"`. Strings gebruik je voor namen, labels, paden, bestandsnamen en alle niet-numerieke data.

`bool`
: Een booleaanse waarde: `True` of `False`. Wordt gebruikt in logica, voorwaarden (`if`) en vergelijkingen.

`list`
: Een geordende verzameling waarden, bijvoorbeeld `[1, 2, 3]` of `["A", "B"]`. Kan verschillende datatypes bevatten en is flexibel, maar minder efficiënt voor wiskundige operaties.

`numpy.ndarray`
: Een numerieke array van vaste grootte en datatype. Dit is het standaard gereedschap voor vector- en matrixberekeningen in de natuurkunde.

Python bepaalt het datatype automatisch, maar je kunt het altijd opvragen met `type(…)`.

Q15

Maak een codecel waarin je voor verschillende variabelen het datatype opvraagt met behulp van `type()`. Gebruik ten minste een `int`, `float`, `str`, `bool`, `list` en een `numpy.ndarray`.

Oplossing:

Voorbeeld – datatype opvragen met `type()`

```python
# Voorbeelden van datatypes en hoe je ze opvraagt

a = 3
b = 3.14
c = "Hallo"
d = True
e = [1, 2, 3]
import numpy as np
f = np.array([1, 2, 3])

print(type(a))   # <class 'int'>
print(type(b))   # <class 'float'>
print(type(c))   # <class 'str'>
print(type(d))   # <class 'bool'>
print(type(e))   # <class 'list'>
print(type(f))   # <class 'numpy.ndarray'>
```